

# สร้างกราฟ Fig 2: RMSD / RMSF / Rg ด้วย matplotlib

บทเรียน matplotlib ฉบับมือใหม่ — สำหรับ combined\_fig2\_dynamics.py

COMFHA — MILK\_FROTHING Project

9 กันยายน 2026

เอกสารนี้เขียนคู่กับ session สอนสด — อ่านตามไปที่ละหัวข้อ แล้วลองเขียนโค้ดเองในไฟล์  
scripts/figures/combined\_fig2\_dynamics.py ตามที่แต่ละหัวข้อบอก

## สารบัญ

|                                                                   |   |
|-------------------------------------------------------------------|---|
| 1 เป้าหมาย: เรากำลังสร้างอะไร                                     | 1 |
| 2 matplotlib พื้นฐาน 5 นาที                                       | 1 |
| 3 ข้อมูลของเรา: ไฟล์ .npz คืออะไร                                 | 2 |
| 3.1 ไฟล์ Rg (results/analysis/blg_rg_CENTER.npz) . . . . .        | 2 |
| 3.2 ไฟล์ RMSD (results/analysis/blg_rmsd_CENTER.npz) . . . . .    | 2 |
| 3.3 ไฟล์ RMSF (results/gate_analysis/rmsf_center.*.npy) . . . . . | 2 |
| 4 เครื่องมือที่มีให้แล้ว: plot_style.py และ utils.py              | 3 |
| 4.1 scripts/plot_style.py . . . . .                               | 3 |
| 4.2 scripts/figures/utils.py . . . . .                            | 3 |
| 5 ลงมือเขียน: โครงไฟล์                                            | 4 |
| 6 ตัวอย่างเต็ม: Rg panel                                          | 5 |
| 7 แบบฝึกหัด: RMSD panel                                           | 5 |
| 8 แบบฝึกหัด: RMSF panel                                           | 6 |
| 9 CAS placeholder panels                                          | 6 |
| 10 ตรวจสอบคำตอบ: ค่าที่ควรได้ (sanity check)                      | 6 |
| 11 ขั้นตอนสุดท้าย: รัน และดูผล                                    | 6 |

## 1. เป้าหมาย: เรากำลังสร้างอะไร

Figure 2 ของ Paper 1 ต้องแสดง โครงสร้างพลศาสตร์ (structural dynamics) ของโปรตีน BLG เทียบกับ CAS ผ่าน 3 ตัวชี้วัด:

- **RMSD** (Root Mean Square Deviation) — โปรตีนเปลี่ยนรูปร่างไปจากจุดเริ่มต้นแค่ไหน ตามเวลา
- **RMSF** (Root Mean Square Fluctuation) — แต่ละ residue สั่นไหวมากน้อยแค่ไหน (เฉลี่ยตลอด trajectory)
- **Rg** (Radius of Gyration) — โปรตีนกระชับ/คลายตัวแค่ไหน ตามเวลา

ฝั่ง BLG มีข้อมูลจริงครบทั้ง 4 replica (CENTER, R1, R2, R3) แล้ว ฝั่ง CAS ยังไม่มีข้อมูล (trajectory กำลัง sync อยู่) — ดังนั้น panel ฝั่ง CAS จะแสดงเป็น ``data pending'' ไปก่อน

ผลลัพธ์สุดท้าย: รูปเดียว ขนาด 3 แถว  $\times$  2 คอลัมน์ (แถว = RMSD/RMSF/Rg, คอลัมน์ = BLG จริง / CAS placeholder) บันทึกเป็นไฟล์ที่ results/figures/paper/

## 2. matplotlib พื้นฐาน 5 นาที

ก่อนแตะโค้ดจริง ต้องเข้าใจ 2 คำนี้ก่อน — เป็นหัวใจของ matplotlib ทั้งหมด:

- **Figure** = ``กระดาน'' ทั้งหมด — เป็นกรอบนอกสุดที่บรรจุทุกอย่าง
- **Axes** = ``กราฟย่อยหนึ่งช่อง'' บนกระดานนั้น — มีแกน x/y ของตัวเอง เส้น จุด legend ของตัวเอง

Figure หนึ่งใบมี Axes ก็ช่องก็ได้ เราต้องการ  $3 \times 2 = 6$  ช่อง สร้างด้วยคำสั่งเดียว:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 2, figsize=(6.85, 7.5))
# axes is a numpy array, shape (3, 2)
# axes[0, 0] = top-left, axes[2, 1] = bottom-right, etc.

axes[0, 0].plot([1, 2, 3], [10, 20, 15]) # draw a line in that one cell only
axes[0, 0].set_xlabel("Time (ns)")
axes[0, 0].set_ylabel("RMSD (nm)")
axes[0, 0].legend()

fig.savefig("test.png") # saves the whole sheet (all cells together)
```

ลองรันโค้ดข้างบนใน Python interactive (หรือไฟล์ทดสอบ) ดูก่อนได้ ให้เห็นภาพว่า axes[i, j] แต่ละช่องคือกราฟแยกกันจริง ๆ

## 3. ข้อมูลของเรา: ไฟล์ .npz คืออะไร

ข้อมูลที่คำนวณไว้แล้วถูกเก็บเป็นไฟล์ .npz (numpy zip) — เปิดด้วย numpy.load() แล้วจะได้ dict-like object ที่ดึงค่าด้วยชื่อ key ได้เลย

### 3.1 ไฟล์ Rg (results/analysis/blg\_rg\_CENTER.npz)

```
import numpy as np
d = np.load("results/analysis/blg_rg_CENTER.npz")
# d has keys: time_ns, rg_nm, rg_mean, rg_std
d["time_ns"] # array, length 10001 -> time at each frame (ns)
d["rg_nm"]   # array, length 10001 -> Rg value at that frame (nm)
d["rg_mean"] # single number -> mean over whole trajectory (should be ~1.496 for
              CENTER)
d["rg_std"]  # single number -> standard deviation
```

### 3.2 ไฟล์ RMSD (results/analysis/blg\_rmsd\_CENTER.npz)

```
d = np.load("results/analysis/blg_rmsd_CENTER.npz")
# key: time_ns, rmsd_backbone, rmsd_sheet, rmsd_helix, rmsd_patch,
#      sheet_resids, helix_resids, patch_resids
d["time_ns"] # length 1001 (note: different length from the Rg file!
              different stride)
d["rmsd_backbone"] # RMSD of the whole-protein backbone
d["rmsd_patch"]   # RMSD of the calyx patch only (should be mean ~0.24 nm)
```

### 3.3 ไฟล์ RMSF (results/gate\_analysis/rmsf\_center.\*.npy)

อันนี้ต่างจาก 2 ไฟล์บน — เก็บเป็น .npy เดี่ยว ๆ (ไม่ใช่ .npz) 2 ไฟล์คู่กัน:

```
resids = np.load("results/gate_analysis/rmsf_center.resids.npy") # (156,) residue
                  numbers 1-156
rmsf    = np.load("results/gate_analysis/rmsf_center.rmsf.npy")  # (156,) RMSF
                  value per residue
# resids[i] always pairs with rmsf[i] -> plt.plot(resids, rmsf) works directly
```

ข้อสังเกตสำคัญ: ชื่อไฟล์ทั้ง 4 replica คือ rmsf\_center, rmsf\_r1, rmsf\_r2, rmsf\_r3 (ตัวพิมพ์เล็ก) ในขณะที่ RMSD/Rg ใช้ CENTER, R1, R2, R3 (ตัวพิมพ์ใหญ่) — ต้องแปลงตัวพิมพ์เองตอน loop ผ่าน 4 replica

## 4. เครื่องมือที่มีให้แล้ว: plot\_style.py และ utils.py

โปรเจกต์นี้มีไฟล์ helper 2 ไฟล์ใน scripts/ ที่ทำให้ไม่ต้องเขียนโค้ดซ้ำ:

### 4.1 scripts/plot\_style.py

```
from plot_style import apply_style, COLORS, double_width, savefig

apply_style() # sets font, spine width, tick size, etc -- same for every figure in
              the project
              # call this once, at the top of the file
```

```

COLORS["center"]      # "#5E4FA2" (purple) -> color for replica CENTER
COLORS["replica1"]    # "#D6604D" (red-orange) -> color for R1
COLORS["replica2"]    # "#2166AC" (blue)
COLORS["replica3"]    # "#1B7837" (green)

double_width          # = 6.85 (inches) -> standard width for a 2-column JCIS figure

savefig(fig, "results/figures/paper/PAPER_FIG2.png") # saves at 300 DPI
                                                         automatically

```

## 4.2 scripts/figures/utils.py

ไฟล์นี้ถูกออกแบบมาเฉพาะสำหรับกราฟเทียบ BLG vs CAS แบบนี้:

```

from utils import SPECIES, run_color, run_label, load_analysis, pending_panel

SPECIES["BLG"]["replicas"] # ["CENTER", "R1", "R2", "R3"] <- loop over this, don't
                             t hardcode!

run_color("R1")              # returns the same color as COLORS["replica1"] (does
                             the lookup for you)
run_label("BLG", "R1")      # returns the string "BLG Replica 1" (use as legend
                             label)

load_analysis("BLG", "rg", "CENTER")
# equivalent to np.load("results/analysis/blg_rg_CENTER.npz") but shorter,
# and a clearer error message if the file doesn't exist

pending_panel(axes[0, 1], "CAS")
# draws a grey box saying "beta-Casein data pending" into that axes cell
# use this for the 3 right-hand cells (RMSD/RMSF/Rg for CAS) since there is no real
# data yet

```

ทำไมต้องใช้ 2 ไฟล์นี้แทนเขียนเอง: ทุกกราฟในเปเปอร์ต้องหน้าตาเหมือนกัน (สี ฟอนต์ ขนาด) ถ้าต่าง-คนต่างเขียนสื่งจะได้กราฟที่สีไม่ตรงกันระหว่าง Figure 1, 2, 3 — utils.py ป้องกัน ปัญหานี้โดยบังคับ-ให้ทุกกราฟดึงสีจากที่เดียวกัน

## 5. ลงมือเขียน: โครงไฟล์

สร้างไฟล์ scripts/figures/combined\_fig2\_dynamics.py เริ่มจากโครงนี้:

```

"""
scripts/figures/combined_fig2_dynamics.py
=====
Figure 2 – structural dynamics (RMSD/RMSF/Rg), BLG real + CAS pending.
"""
from pathlib import Path
import sys

import numpy as np
import matplotlib.pyplot as plt

```

```

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "scripts"))
from plot_style import apply_style, COLORS, double_width, savefig
from utils import SPECIES, run_color, run_label, load_analysis, pending_panel

apply_style()

fig, axes = plt.subplots(3, 2, figsize=(double_width, 7.5))
# row 0 = RMSD, row 1 = RMSF, row 2 = Rg
# column 0 = BLG, column 1 = CAS (pending)

# ===== TODO: Rg panel (axes[2, 0]) =====

# ===== TODO: RMSD panel (axes[0, 0]) =====

# ===== TODO: RMSF panel (axes[1, 0]) =====

# ===== TODO: CAS pending panels (axes[0,1], axes[1,1], axes[2,1]) =====

fig.tight_layout()
out = ROOT / "results" / "figures" / "paper" / "PAPER_FIG2_DYNAMICS.png"
savefig(fig, out)

```

ทำความเข้าใจ `sys.path.insert` บรรทัดนั้นก่อน: ไฟล์นี้อยู่ที่ `scripts/figures/combined_fig2_dynamics.py` แต่ `plot_style.py` และ `utils.py` อยู่ที่ `scripts/` (สูงกว่าหนึ่งชั้น) Python จะหาไฟล์ import ไม่เจอ ถ้าไม่บอก path ตรง ๆ — `parents[2]` หมายถึง ``ถอยขึ้นไป 2 ชั้นจากไฟล์นี้" ซึ่งจะได้ root ของโปรเจกต์ (MILK\_FROTHING/) แล้วค่อยต่อ / "scripts"

## 6. ตัวอย่างเต็ม: Rg panel

นี่คือ panel เดียวที่จะเขียนให้ดูครบเป็นตัวอย่าง — ที่เหลือ (RMSD, RMSF) ให้ลองเขียนเองในหัวข้อถัดไป โดยเทียบโครงสร้างกับอันนี้

```

ax = axes[2, 0]
for run in SPECIES["BLG"]["replicas"]:          # ["CENTER", "R1", "R2", "R3"]
    d = load_analysis("BLG", "rg", run)
    ax.plot(d["time_ns"], d["rg_nm"],
            color=run_color(run), lw=0.8, alpha=0.85,
            label=run_label("BLG", run))
ax.set_xlabel("Time (ns)")
ax.set_ylabel(r"$R_g$ (nm)")
ax.legend(loc="upper right", ncol=2)
ax.set_title("BLG", fontsize=9)

```

อ่านทีละบรรทัด:

1. `for run in SPECIES["BLG"]["replicas"]:` — loop ผ่านชื่อ 4 replica โดยไม่ hardcode list เอง (ถ้าวันหนึ่งเพิ่ม replica R4 โค้ดนี้จะรองรับอัตโนมัติ ไม่ต้องแก้)
2. `load_analysis("BLG", "rg", run)` — โหลดไฟล์ .npz ของ replica นั้น

3. `ax.plot(x, y, ...)` — วาดเส้นหนึ่งเส้นในช่องกราฟนี้ ทำซ้ำ 4 ครั้ง (4 replica) จะได้ 4 เส้นสีต่างกันซ้อนกันในช่องเดียว
4. `run_color(run)` — ไม่ต้องเขียน `if/else` เลือกสีเอง ให้ `utils.py` จัดการ
5. `ax.legend(...)` — เรียกครั้งเดียวหลัง `loop` จบ (ไม่ใช่ในลูป) เพราะแต่ละ `ax.plot` ใส่ label ของตัวเองไว้แล้ว `legend` จะไปเก็บมารวมทีเดียว

ทดสอบ: รันแค่ส่วนนี้ก่อน (คอมเมนต์ TODO อื่นไว้ก่อน) แล้วเปิดไฟล์ PNG ที่ได้ดู ควรเห็นเส้น 4 สีที่เริ่มสูง ( $R_g \approx 3$  nm สำหรับ AlphaFold start ถ้าเป็น CAS แต่ที่นี่คือ BLG ควรจะแบนราบ  $\sim 1.5$  nm ตลอด — ดู Section 10 ประกอบ)

## 7. แบบฝึกหัด: RMSD panel

ลองเขียน `axes[0, 0]` เอง โดยใช้โครงเดียวกับ `Rg` panel ด้านบน แต่เปลี่ยน:

- `kind` จาก `"rg"` เป็น `"rmsd"`
- `key` ของ `x` คือ `"time_ns"` เหมือนเดิม, แต่ `key` ของ `y` คือ `"rmsd_backbone"` (ไม่ใช่ `"rg_nm"`)
- `ylabel` ควรเป็น `"RMSD (nm)"`

คำใบ้เรื่อง `patch`: ไฟล์ RMSD มีทั้ง `rmsd_backbone` และ `rmsd_patch` — ถ้าอยากโชว์ทั้งคู่ในช่องเดียว ต้อง plot 2 เส้นต่อ replica (backbone เส้นทึบ, patch เส้นประ) ลองดูว่า `ax.plot(..., ls="--")` วาดเส้นประยังงัย แล้วลองใส่ `alpha` ต่างกันสำหรับ backbone vs patch เพื่อไม่ให้ 8 เส้น (4 replica  $\times$  2 region) รกเกินไป

เขียนเสร็จแล้วรันดู ก่อนไปข้อถัดไป — ถ้า error ให้อ่าน `traceback` บรรทัดสุดท้ายก่อน มักจะบอกตรงๆ ว่า `key` โหนพิมพ์ผิด หรือไฟล์ไหนหาไม่เจอ

## 8. แบบฝึกหัด: RMSF panel

อันนี้ต่างจาก 2 อันบนเล็กน้อย เพราะ RMSF ไม่ได้อยู่ใน `load_analysis` (มันเป็น `.npy` เดี่ยว ไม่ใช่ `.npz` ที่ `utils.py` รองรับ) ต้องโหลดตรงด้วย `np.load` เอง:

```
GATE_DIR = ROOT / "results" / "gate_analysis"

# filenames use lowercase, and "center" not "CENTER" -- convert it yourself
_RMSF_TAG = {"CENTER": "center", "R1": "r1", "R2": "r2", "R3": "r3"}
```

โจทย์: เขียน `loop` คล้ายเดิม แต่แกน `x` คือ `resids` (เลข residue 1-156) แทนเวลา เพราะ RMSF ไม่ใช่ `time series` — มันคือ ``แต่ละตำแหน่งในโปรตีนสั่นมากแค่ไหน (เฉลี่ยทั้ง trajectory)`` ดังนั้น `xlabel` ควรเป็น `"Residue number"` ไม่ใช่ `"Time (ns)"`

คำใบ้: ใช้ `_RMSF_TAG[run]` แปลงชื่อ `run` ก่อนต่อ path เช่น `GATE_DIR / f"rmsf_{_RMSF_TAG[run]}.rmsf"`

## 9. CAS placeholder panels

ส่วนนี้เขียนให้เสร็จเลย ง่ายกว่า 3 อันบนมาก — เรียก `pending_panel` 3 ครั้งกับ 3 ช่องขวา:

```
for row in range(3):
    pending_panel(axes[row, 1], "CAS")
axes[0, 1].set_title("CAS  $\beta$ (-Casein)", fontsize=9)
```

## 10. ตรวจสอบคำตอบ: ค่าที่ควรได้ (sanity check)

ก่อนเช็กรูปที่ได้ ให้เทียบกับตัวเลขที่ยืนยันแล้วใน CLAUDE.md:

| ตัวชี้วัด          | Replica     | ค่าที่ควรได้                                               |
|--------------------|-------------|------------------------------------------------------------|
| Rg (mean)          | CENTER      | $1.496 \pm 0.009$ nm (เส้นควรแบนราบตลอด ไม่มี trend)       |
| RMSD (patch, mean) | ทุก replica | $\approx 0.24$ nm (แบนราบ ไม่ค่อยขึ้น)                     |
| RMSD (patch, R1)   | R1          | สูงกว่าอันอื่นเล็กน้อย (0.347 nm) — เป็นของจริง ไม่ใช่ bug |

ถ้าเส้น Rg ของคุณพุ่งขึ้นเรื่อย ๆ ไม่หยุด หรือค่า mean ต่างจาก 1.496 nm มาก (เช่น 3.0 nm) แปลว่าโหลดไฟล์ผิด replica หรือพล็อตแกนสลับ — กลับไปเช็ค Section 3 ก่อน

## 11. ขั้นตอนสุดท้าย: รัน และดูผล

```
cd ~/Workspace/MILK_FROTHING
/Users/mac2022-1/opt/anaconda3/envs/research-env/bin/python3 \
  scripts/figures/combined_fig2_dynamics.py
```

ถ้ารันผ่านไม่มี error จะได้ไฟล์ results/figures/paper/PAPER\_FIG2\_DYNAMICS.png เปิดดูด้วย Preview หรือลาก path ไปให้ Claude ช่วยดูก็ได้ (ถามใน session ต่อได้เลย ``ช่วยดูรูปนี้หน่อย" พร้อม path)

เมื่อเขียนเสร็จและรันผ่านแล้ว ให้เอาโค้ดกลับมาให้ Claude รีวิวใน session — จะเช็คให้ว่าตรงกับ convention ของโปรเจกต์ใหม่ (การตั้งชื่อไฟล์ output, ที่เก็บไฟล์, ว่า RMSD/RMSF/Rg ตรงกับตัวเลขที่ล็อกไว้หรือเปล่า) ก่อนจะ commit